Testing Policy Document

COS 301 - Capstone 2026
Capstone Project: EquityLens
By The Big Five

Project Name: EquityLens
Client: Amazon Web Services (AWS)
Team: The Big Five (TB5)

Team members:

- Abdulrahman Sabah (Team Lead) - u24566170
- Joshua Heath - u23541475
- Antony van Straten - u24590739
- Jonty Honey - u23536862

Contents

10. Policy Enforcement and Review

1. Introduction

1.1 Purpose

This policy defines the standards, procedures and responsibilities for all software testing on EquityLens. It establishes a consistent approach to planning, executing, documenting and reviewing testing activities.

The policy covers testing objectives, the types of testing performed and how each is carried out, the tools and environments used, how test data is managed, how defects are reported and resolved, the acceptance criteria that gate merges and demos, and the roles of everyone involved in the testing lifecycle.

1.2 Scope

The policy applies to the entire mono-repository: the React frontend (frontend/), the FastAPI backend (backend/), database migrations (backend/alembic/), and the CI pipeline that binds them (.github/workflows/ci.yml). It governs both automated testing and the manual verification procedures used where automation is not yet practical.

2. Testing Objectives

- Verify functional correctness. Every functional requirement in the SRS is exercised by at least one automated test at the appropriate level - unit for calculations and components, integration for API behaviour and end-to-end for user journeys.
- Protect the user's data. Authentication enforcement and per-user data scoping are treated as the highest-priority test subjects: integration tests must prove that unauthenticated requests are rejected and that no query can return another user's data.
- Prove financial calculations. Each of the seven indicators has a dedicated unit test module asserting known input/output pairs, because a plausible-looking wrong number is worse than an error in an analytics product.
- Prevent regressions. Bug fixes shipswith a test that reproduces the defect, so a bug fixed once cannot silently return.
- Keep main deployable. The full pipeline runs on every push and pull request; a red pipeline blocks the merge, so the deployable branch is protected mechanically rather than by discipline.
- Make quality visible. Coverage and test results are published on every CI run.

3. Acceptance Criteria and Quality Gates

3.1 Per Pull Request (merge criteria)

A pull request may merge only when all of the following hold:

- All CI jobs pass. frontend lint, unit and E2E tests, backend tests, security scans, and migration reversibility.
- New or changed behaviour is covered by new or updated tests.
- assert the new behaviour.

3.2 Per Feature

A feature is complete for demo purposes only when it is verified at every applicable level: unit tests for its logic, integration tests for its API surface, an end-to-end test for its user journey, and no mocked data in the demonstrated path. This mirrors the Demo 2 requirement that demonstrated use cases be fully implemented with full integration, unit and E2E testing.

3.3 Per Demo

Before each demo: main is green, all end-to-end journeys pass against the demo build, all demonstrated features satisfy, and known remaining defects are recorded as GitHub Issues so nothing surprising is discovered live.

4. Testing Types and Procedures

4.1 Static Analysis and Code Quality

The first quality gate runs before any test executes:

- ESLint (npm run lint) enforces the frontend ruleset on every push and PR in CI.
- ruff (ruff check .) is the standard Python linter, run locally before every push.
- npm audit (high severity, production dependencies) and pip-audit scan dependencies for known vulnerabilities on every CI run.
- Bandit scans the backend application code for insecure patterns on every CI run.
- Trivy scans the repository filesystem for vulnerabilities, misconfigurations and accidentally committed secrets on every CI run.

4.2 Unit Testing

Frontend. Vitest with React Testing Library, jest-dom matchers and user-event, running in a jsdom environment (configured in vite.config.js with a shared setup file). Tests live beside the component they test. Components are tested through their rendered behaviour, pure helpers are tested directly as functions.

Backend. pytest modules under backend/tests/unit/, one module per subject. Indicator modules assert known input/output pairs, service tests isolate external dependencies with unittest.mock/pytest-mock patches so no unit test ever performs network I/O.

4.3 Integration Testing

Backend integration tests under backend/tests/integration/ exercise complete API flows through FastAPI's TestClient against the real application object. The shared fixtures in conftest.py provide an isolated in-memory SQLite database created fresh per test and destroyed afterwards, dependency overrides for get_db and get_current_user so flows run as a known authenticated user without contacting the identity provider, reusable sample data fixtures so tests share realistic and consistent inputs.

Integration tests assert 401 responses for unauthenticated requests and verify that user-scoped queries return only the fixture user's data. In CI the backend job runs against a real PostgreSQL 16 service container so SQLite conveniences cannot mask PostgreSQL behaviour differences in the migration and query paths.

4.4 End-to-End Testing

Playwright drives the application in a browser from frontend/e2e/ configured to start the dev server automatically (playwright.config.js webServer): npm run test:e2e. Journeys use shared authentication helpers under e2e/helpers/ so journeys log in the same way the user does. E2E specs assert what the user experiences, never internal implementation details.

4.5 Migration Testing

Every CI run executes alembic upgrade head, a full downgrade then upgrade again versus the PostgreSQL 16 service. This proves every schema migration is reversible before it can merge.

4.6 Non-Functional Verification

Each of the five SRS quality requirements has a defined verification method. Where verification is not yet automated, that is stated honestly rather than implied:

- Security (NFR-1): Authentication rejection and user-scoping integration tests, plus Bandit, pip-audit, npm audit and Trivy scanning in every CI run. Secrets hygiene is verified by Trivy's secret scanner.
- Performance (NFR-2): currently verified manually, performance is not ideal for demo 2, this will addressed going forward into demo 3
- Reliability (NFR-3): Unit and integration tests assert graceful degradation. Uptime measurement activates with the public deployment via an external monitor against /health.
- Scalability (NFR-4): No scalability tests have been run nor has sufficient preparation been done for scalability to be in the scope of demo 2.
- Usability & Accessibility (NFR-5): verified per feature in review. We as of now cannot guarantee that this application satisfies full usability and accessibility standards

5. Tools and Environments

The testing toolchain, all pinned in the repository's dependency manifests:

- Vitest with @vitest/coverage-v8 - frontend test runner and coverage.
- React Testing Library (+ jest-dom, user-event) - component testing through user-visible behaviour.
- Playwright (@playwright/test) - browser end-to-end testing.
- pytest with pytest-cov, pytest-asyncio, pytest-mock and httpx - backend unit and integration testing against the FastAPI TestClient.
- Codecov - coverage publication from both suites on every CI run.
- ESLint, ruff, Bandit, pip-audit, npm audit, Trivy - static analysis layer.

Environments:

- Local development - the Docker Compose environment (frontend :5173, backend :8000, PostgreSQL 16 :5432); both test suites also run directly on the developer machine (`npm run test`, `python -m pytest`) without any services for the unit level.
- Continuous integration - GitHub Actions on Ubuntu runners: Node 20 for the frontend job, Python 3.11 plus a PostgreSQL 16 service container for the backend job. Every push and pull request to `main` and `dev` runs the full matrix; concurrent runs on the same ref are cancelled in favour of the newest commit.
- Staging and production - once the AWS deployment is live (SAS Section 5), the E2E suite gains a smoke-test run against staging, and uptime monitoring begins against production.

6. Test Data Management

- All test data is synthetic. Fixtures define sample users and portfolio payloads; no real personal or brokerage data ever enters a test, a fixture, or a repository file.
- Backend tests build their entire database from the SQLAlchemy metadata per test and destroy it afterwards; no test depends on pre-existing state, execution order, or another test's leftovers.
- External services (market data, news, Bedrock, Cognito) are never called from unit or integration tests - they are patched at the service boundary, both for determinism and so the suite runs offline and free.
- E2E tests create what they need through the UI itself (registering a fresh user per run via the shared helpers), which doubles as a continuous test of the registration path.
- Statement-import tests use fabricated statement fixtures that mirror the EasyEquities format without containing any real account's data.

7. Defect Management

Defects are managed in GitHub Issues, in the same repository as the code, so a defect's history and its fix are permanently linked.

Severity levels, assigned at triage:

- Critical - security failures (cross-user data exposure, authentication bypass), data corruption, or the application failing to start. Work stops on the affected area until resolved; a critical defect blocks any demo of that feature.
- Major - a feature is broken or materially wrong (an indicator calculating incorrectly, an import silently dropping holdings). Scheduled into the current sprint.
- Minor - cosmetic or low-impact issues (spacing, copy, non-blocking console warnings). Batched and scheduled around feature work.

Lifecycle: a defect is reported as an issue with reproduction steps, expected and actual behaviour; triaged and assigned to the owner of the affected area; fixed on a fix/[description]/[INITIALS] branch whose pull request contains a regression test reproducing the defect; and closed by the merge, with the issue referencing the PR. The regression-test requirement is non-negotiable - it is the mechanism that converts every bug into a permanent guarantee.

8. Roles and Responsibilities

- The author of a change writes its tests in the same pull request, at every level the change touches. Testing is not a separate phase or a separate person's job.
- The two reviewers verify the tests assert real behaviour, cover the failure paths, and follow the naming and placement conventions. Approving a PR means co-owning its quality.
- The area owner (per the ownership map in the project's working agreement , the owner of the defects triages defects in their area and maintains that area's test suites as the code evolves.
- The team lead (Abdulrahman Sabah) owns this policy's enforcement: pipeline health, unresolved-defect review in the twice-weekly stand-ups, and the release criteria check before each demo.
- The whole team owns `main`: anyone who sees a red pipeline treats restoring it as their highest-priority task regardless of who broke it.

9. Test Reporting and Current Status

Reporting mechanisms: every CI run reports per-job pass/fail in the pull request; coverage is uploaded to Codecov from both suites (frontend lcov, backend XML) on every run; locally, `pytest` prints per-file line coverage with missing-line detail (`term-missing`) and `npm run coverage` produces the equivalent frontend report.

10. Policy Enforcement and Review

This policy is enforced by machinery rather than memory: the CI pipeline implements Sections 4.1-4.5 on every push, and the two-reviewer process implements the judgement-based rules the pipeline cannot check. The policy itself lives in the repository and changes only by pull request, reviewed like the code it governs. It is revisited at each demo boundary.